1 Foundational Concepts

This chapter establishes the theoretical and technical vocabulary required to understand the TRACE (Technical Resolution and Claims Evaluation) system. Each concept presented here is directly employed in the system implementation and is referenced explicitly in the subsequent chapters on System Design, Implementation, and Evaluation.

TRACE is a hybrid intelligent pipeline that combines a deterministic rule engine, a cascaded pair of XGBoost classifiers, an optional Large Language Model (LLM) layer with dual-provider support (OpenAI and OpenRouter), and an OCR-based image scanning module. Together, these components analyse automotive ECU warranty claims consisting of a Diagnostic Trouble Code (DTC), technician's free-text notes, and a voltage reading, and produce a structured decision comprising a failure analysis category, a warranty decision, a confidence score, and a human-readable justification.

1.1 System Evolution

TRACE has evolved through several major architectural milestones, as documented in the project's git history:

Milestone	Description
Initial Commit	Basic ML predictor with RandomForest and rule engine
LLM Integration	OpenRouter API added for technician note categorization with retry logic
Cascade Architecture	Failure Analysis classifier output used as features for Warranty Decision classifier
Logging Infrastructure	Centralized logging with DecisionLogger across all pipeline stages
Dataset Upgrade v9	Synthetic dataset expanded to 100,000 rows (2019–2025) with 93–96% pattern correlation
Voltage Rules Added	over_voltage (>16V) and low_voltage (<11V) rules added to rule engine (7 → 9 rules)
EasyOCR Integration	Image scanning endpoint for extracting DTC codes and voltage from technician photos
RandomForest → XGBoost	ML model replaced with XGBoost (1000 estimators) for superior accuracy on structured+text features
Dual LLM Providers	OpenAI (gpt-4o-mini) added as primary provider with OpenRouter as fallback
LLM Consistency	Temperature set to 0, seed=42, disambiguation rules added for deterministic outputs
LLM Confidence Weight	LLM signal integrated into score combination with 15% weight
Feature Engineering	Added voltage_bracket, dtc_count_bracket, and interaction features (volt_high_and_P, etc.)

The sections below describe the **current** system state. Historical implementations (e.g., RandomForest, 7-rule engine) are noted where relevant for comparison.

2 Machine Learning Overview

2.1 Supervised Learning

Supervised learning is the branch of machine learning in which a model is trained on a labelled dataset that is, a dataset where each input example is paired with a known output

value. The model learns a mapping function f from the input feature space X to the output label space Y such that, for an unseen input x, the predicted output $f(x)$ approximates the true label y with minimum error.

TRACE uses supervised learning throughout. The training dataset is `synthetic_warranty_claims_v9.csv`, containing 100,000 synthetic warranty claim records spanning 2019–2025. Each record carries both a Failure Analysis label and a Warranty Decision label, making it a supervised, multi-output classification problem. The dataset is synthetically generated with 93–96% pattern correlation to mimic real-world noise — patterns like "moisture keywords → Customer Failure" hold approximately 91–95% of the time, not always.

Used in: Section 4.2 (Dataset Construction), Section 5.1 (Model Training)

2.2 Multi-Class Classification

A classification task assigns a discrete class label to each input. When the number of possible classes is greater than two, the problem is called multi-class classification. Both classifiers in TRACE are multi-class:

- **Failure Analysis classifier:** 6 output classes — NTF, Sensor short due to moisture, Connector damage, EOS (Electrical OverStress), ASIC CJ327 failure due to EOS, Controller failure due to supplier production failure.
- **Warranty Decision classifier:** 3 output classes — Production Failure, Customer Failure, According to Specification.

Historical note: Earlier versions of TRACE used 9 FA classes. The v9 dataset consolidation reduced this to 6 well-defined categories for improved model accuracy and reduced confusion between similar failure modes.

Each classifier returns a probability vector over all its classes, and the class with the highest probability is selected as the prediction. This probability vector also plays a structural role in the cascade architecture described in Section 3.8.3.

Used in: Section 5.2 (Cascade Architecture)

3 Core Algorithms

3.1 Decision Trees

A decision tree is a hierarchical model that partitions the input space through a sequence of binary questions (splits). Each internal node tests a single feature against a threshold; each leaf node holds a class label (for classification) or a numeric value (for regression). The tree is grown by recursively selecting the split that maximises an information-theoretic criterion.

The two most common split criteria for classification are:

$$G(\text{node}) = 1 - \sum_k p_k^2 \tag{3.1}$$

$$H(\text{node}) = -\sum_k p_k \log_2(p_k) \tag{3.2}$$

where p_k is the proportion of class k examples at the node.

Splits that minimise Gini impurity (or equivalently maximise information gain) are preferred. A node is declared a leaf when a stopping criterion is met: maximum depth reached, fewer than `min_samples_split` examples remain, or no split improves purity.

Decision trees are interpretable and require no feature scaling, but they overfit readily on training data. Both ensemble methods described next — Random Forest (Section 3.2) and XGBoost (Section 3.3) — address this limitation, albeit through fundamentally different strategies: bagging versus boosting.

Used in: Section 3.2 (Random Forest uses Decision Trees as base learners), Section 3.3 (XGBoost uses Decision Trees as base learners)

3.2 Random Forest

Random Forest is an ensemble method that reduces overfitting by training many decision trees independently and aggregating their predictions. Each tree is trained on a bootstrap sample of the training data, and at each split only a random subset of features is considered. The final prediction is determined by majority vote across all trees.

Historical Role in TRACE: The original TRACE implementation used `RandomForestClassifier` (scikit-learn) with 200 estimators for both the Failure Analysis and Warranty Decision classifiers. This was the production model until commit `a518371`, when it was replaced by XGBoost (Section 3.3). The RandomForest implementation is archived in `ml_predictor_DecisionTree.py` for reference.

3.2.1 Bagging and Bootstrap Aggregation

Bagging (Bootstrap Aggregation) trains each tree on an independently drawn bootstrap sample of the original dataset. A bootstrap sample is formed by sampling n rows from the training set with replacement, where n is the original dataset size. On average, approximately 63.2% of unique rows appear in any given bootstrap sample; the remaining 36.8% are "out-of-bag" (OOB) and can be used for internal validation without a separate validation set.

For classification, the final prediction is determined by a majority vote across all trees:

$$\hat{y} = \arg\max_{k} \sum_{t=1}^{T} I(\hat{y}_t = k) \tag{3.3}$$

where T is the number of trees, $\hat{y}_t$ is the prediction of tree t, and $I(\cdot)$ is the indicator function.

Aggregating over many trees that were trained on different bootstrap samples reduces variance without significantly increasing bias — the key trade-off that makes Random Forest a robust general-purpose classifier.

3.2.2 Random Feature Subsampling

At each node split during tree construction, only a random subset of m features is considered as candidates for the best split. This decorrelates the trees: even if one feature is highly predictive, it will not dominate every tree. The standard recommendation for classification is:

$$m = \sqrt{p} \tag{3.4}$$

where p is the total number of features.

In `scikit-learn`, this corresponds to `max_features="sqrt"`, which is the default. Feature subsampling ensures that different trees specialise in different parts of the feature space, increasing diversity and reducing ensemble variance.

Hyperparameter	Value / Role in TRACE (Retired)
`n_estimators`	200 — number of trees in each forest. Higher values reduce variance but increase training time.
`random_state`	42 — seed for reproducibility of bootstrap samples and feature selection.
`n_jobs`	-1 — all available CPU cores used in parallel for tree construction.
`class_weight`	`"balanced"` on `clf_wd` — reweights classes inversely proportional to frequency; compensates for imbalanced warranty decision labels.
`max_depth`	None (default) — trees grow until leaves are pure or `min_samples_split` is reached.

3.2.3 Hyperparameters (Historical TRACE Configuration)

Used in: Section 3.3 (XGBoost superseded RandomForest)

3.3 XGBoost

TRACE currently uses XGBoost (`XGBClassifier`) for both the Failure Analysis and Warranty Decision classifiers, replacing the original RandomForest configuration described in Section 3.2. This section introduces the gradient boosting framework, explains the motivation for the migration, and documents the tuned hyperparameters used in production.

3.3.1 Gradient Boosting

Gradient boosting is an ensemble method that builds trees *sequentially*: each new tree is trained to correct the residual errors of the combined ensemble so far. Formally, let $F_t(x)$ be the ensemble prediction after t trees. The $t+1$-th tree $h_{t+1}(x)$ is fitted to the negative gradient of the loss function evaluated at $F_t(x)$:

$$h_{t+1}(x) = \arg\min_{h} \sum_{i=1}^{n} L\left(y_i, F_t(x_i) + h(x_i)\right) \tag{3.5}$$

where L is the differentiable loss function (e.g., logistic loss for classification). The ensemble is updated by:

$$F_{t+1}(x) = F_t(x) + \eta \cdot h_{t+1}(x) \tag{3.6}$$

where η is the learning rate (also called shrinkage), a small positive constant that controls the contribution of each tree. A low learning rate (e.g., 0.02) requires more trees but typically yields better generalisation.

The key distinction from Random Forest (Section 3.2) is that gradient boosting trees are **additive** (each tree corrects the ensemble's remaining error) rather than **independent** (each tree is trained on a bootstrap sample and votes). This makes boosting more prone to overfitting if unregularised, but also more powerful when properly tuned.

3.3.2 Why XGBoost Replaced RandomForest

The migration from RandomForest to XGBoost was motivated by three empirical findings documented in `docs/xgboost-implementation-plan.md` and `docs/xgboost-tuning-results.md`:

1. **Superior accuracy on structured+text features.** XGBoost handles the mix of sparse TF-IDF features and dense one-hot encoded features more effectively than

RandomForest. On the TRACE dataset (100,000 synthetic claims with 90 engineered features), XGBoost achieved 89.0% WD accuracy compared to RandomForest's 86.4%.

2. **Better-calibrated probabilities.** The cascade architecture (Section 3.4) feeds FA classifier probabilities as features to the WD classifier. XGBoost produces smoother, better-calibrated probability distributions than RandomForest, which tends to produce overconfident (near-0/1) probabilities that degrade cascade performance.

3. **Built-in regularisation.** XGBoost's L1 and L2 regularisation terms (`reg_alpha` and `reg_lambda`) directly penalise complex trees, reducing overfitting on the synthetic dataset without requiring separate pruning or early stopping. In TRACE, `reg_lambda=0.1` is applied to all trees.

The migration was carried out in commit `a518371` ("Replace RandomForest with XGBoost and add voltage feature to ML model"), followed by hyperparameter tuning in commit `98b61ae`. The RandomForest implementation is archived in `ml_predictor_DecisionTree.py`.

3.3.3 Hyperparameters in TRACE

Hyperparameter	Value / Role in TRACE
`n_estimators`	1000 — number of boosting rounds. Higher values enable convergence with low learning rate.
`max_depth`	10 — deep enough to capture complex feature interactions without excessive overfitting.
`learning_rate`	0.02 — low learning rate for stable convergence with many trees.
`min_child_weight`	3 — minimum sum of instance weight needed in a child; prevents overfitting on small leaf nodes.
`subsample`	0.8 — row subsampling ratio; each tree sees 80% of training rows, adding robustness.
`colsample_bytree`	0.8 — column subsampling ratio; each tree considers 80% of features, reducing correlation.
`reg_lambda`	0.1 — L2 regularisation on leaf weights; prevents overfitting on synthetic data.
`random_state`	42 — seed for reproducibility.
`n_jobs`	-1 — all available CPU cores used in parallel for tree construction.
`eval_metric`	`mlogloss` — multi-class log loss for monitoring convergence.

Used in: Section 5.2 (Model Training), Section 3.4 (Cascade Architecture)

3.3.4 Tuning Results and Accuracy Ceiling

Fifteen or more hyperparameter configurations were evaluated, varying tree count, depth, learning rate, subsample ratio, and regularisation. The best configuration (`n_estimators=1000`, `max_depth=10`, `learning_rate=0.02`) achieved a WD accuracy ceiling of 89.0%.

Key findings from the tuning process:

- **Voltage feature (+1.9%):** Adding voltage as a scaled numeric feature and voltage-bracket OHE improved accuracy from 86.4% to 88.3%.
- **XGBoost replacement (+0.5%):** Switching from RandomForest to XGBoost improved accuracy from 88.3% to 88.8%.

- **Hyperparameter tuning (+0.2%):** Grid search over 15 configurations pushed accuracy to 89.0%.
- **Ceiling analysis:** Customer Failure recall (83%) is the bottleneck. Approximately 1,227 Customer Failure samples are misclassified as Production Failure due to overlapping DTC patterns and statistically identical voltage distributions in the ASIC CJ327 failure category.
- **What did not work:** Sample weighting (hurts overall accuracy), wider/deeper trees (overfits), increased TF-IDF dimensions (no gain), and voltage × DTC interaction features (minimal gain beyond the existing interactions).

Used in: Section 5.2 (Model Training), Section 5.5 (Results Discussion)

3.4 Cascade Classification Architecture

TRACE employs a cascade architecture in which the output of the first classifier — the Failure Analysis (FA) classifier — is used as additional input features to the second classifier — the Warranty Decision (WD) classifier. The motivation is that the warranty decision is causally dependent on the failure analysis: whether a part failure was caused by a production defect or by customer misuse directly determines whether the warranty claim is approved or rejected.

The cascade is implemented as follows:

1. The FA classifier produces a probability vector over all 6 failure analysis classes for each training example.
2. This probability vector is horizontally concatenated onto the original feature matrix to form the augmented feature matrix X_{wd}.
3. The WD classifier is trained on X_{wd}, giving it explicit access to the FA model's belief distribution.

$$\begin{aligned} X_{\text{base}} = [&X_{\text{complaint}} \mid X_{\text{dtc\_tfidf}} \mid X_{\text{dtc\_flags}} \mid \\ &X_{\text{supplier}} \mid X_{\text{mileage}} \mid X_{\text{year}} \mid \\ &X_{\text{mileage\_bracket}} \mid X_{\text{claim\_age}} \mid X_{\text{voltage\_bracket}} \mid \\ &X_{\text{dtc\_count\_bracket}} \mid X_{\text{interactions}}] \end{aligned} \tag{3.7}$$

$$\text{fa\_probs} = \texttt{clf\_fa.predict\_proba}(X_{\text{base}}) \tag{3.8}$$

$$X_{\text{wd}} = [X_{\text{base}} \mid \text{fa\_probs}] \tag{3.9}$$

Critical implementation detail — Out-of-Fold (OOF) probabilities: Naively training `clf_fa` on all of X_{tr} and then using its in-sample probabilities to train `clf_wd` would introduce data leakage. TRACE avoids this by using 5-fold cross-validation to generate OOF FA probabilities for the training set, ensuring that the probabilities passed to `clf_wd` during its training come from a held-out fold that `clf_fa` was not trained on. The final `clf_fa` is then retrained on all of X_{tr} for inference.

ML confidence calculation: The ML confidence is computed as the geometric mean of the top-class probabilities from both classifiers:

$$\text{ml\_confidence} = \sqrt{\text{FA\_top\_prob} \times \text{WD\_top\_prob}} \times 100 \tag{3.10}$$

clamped to the range [0, 98].

Historical note: The cascade architecture was originally implemented with RandomForest classifiers. It was migrated to XGBoost in commit `a518371` with tuned hyperparameters (1000 estimators, max_depth=10, learning_rate=0.02) in commit `98b61ae`.

Used in: Section 5.3 (OOF Cascade Training), Section 3.8.3 (Cross-Validation)

4 Rule-Based Systems

4.1 What is a Rule Engine?

A rule-based system (rule engine) encodes domain knowledge as a set of explicit IF–THEN logical rules. Each rule specifies a condition (a predicate over input values) and a consequence (an action or output to produce when the condition is satisfied). Rule engines are deterministic: the same input always produces the same output, and the reasoning process is fully transparent and auditable.

In the TRACE pipeline, the rule engine serves as the first line of classification for clear-cut cases. Rules encode automotive warranty expertise that would take an ML model many examples to learn and even then, the model's reasoning would be opaque. By handling high-confidence edge cases (such as moisture damage or No-Trouble-Found claims) with explicit rules before ML scoring, TRACE improves overall accuracy, reduces the ML model's burden, and produces reasoning strings that are directly interpretable by warranty analysts.

4.2 Priority-Based Rule Evaluation

TRACE implements a priority-ordered rule evaluation strategy. The nine rules are evaluated in a fixed sequence; the first rule whose condition evaluates to `True` fires, and its associated output is returned immediately — no subsequent rules are evaluated. This is sometimes called a "first-match" or "ordered" rule set.

Rule ID	Trigger Condition	Warranty Decision	Confidence
over_voltage	voltage > 16.0V	Customer Failure	93.0%
low_voltage	voltage < 11.0V	Customer Failure	95.0%
moisture	Keywords: water, moisture, wet, flood, rain, humid, corrosion, corroded	Customer Failure	91.0%
physical_damage	Keywords: crack, broken, impact, collision, bent, misuse, dropped, physical damage	Customer Failure	88.5%
ntf	Keywords: no fault, ntf, no trouble, no issue, no defect, intermittent, cannot reproduce	According to Spec	95.0%
u_code	DTC regex: `\bU[0-9A-Fa-f]{4}\b` (CAN/LIN fault)	Production Failure	57.0%
p_code_engine	P0-series DTC + engine symptom keywords (jerk, pickup, acceleration, overheat, fuel, idle, rough)	Production Failure	80.5%
c_code	DTC regex: `\bC[0-9A-Fa-f]{4}\b` (chassis fault)	Production Failure	80.0%
b_code	DTC regex: `\bB[0-9A-Fa-f]{4}\b` (body fault)	Production Failure	80.0%

Historical note: The original rule engine had 7 rules (moisture through b_code). Voltage rules (over_voltage, low_voltage) were added as priorities 1 and 2 because they

are the most objective and reliable indicators — voltage is a direct measurement, not a keyword match.

Design rationale: Voltage rules come first because they are unconditional measurements. Keyword rules (moisture, physical damage, NTF) come next because they are high-confidence pattern matches. DTC prefix rules come last because they have lower confidence and are more ambiguous.

The confidence values associated with rules are not computed from data; they are calibrated domain estimates reflecting the expected precision of each rule when applied to realistic automotive warranty claims.

4.3 Confidence Thresholding and Score Combination

TRACE does not output the rule confidence directly. Instead, the rule confidence and the ML model confidence are blended through a weighted combination scheme to produce a single unified confidence score. This score is then compared against two fixed thresholds:

$$\texttt{CONFIDENCE\_THRESHOLD\_FIRM} = 85.0 \tag{4.1}$$

$$\texttt{CONFIDENCE\_THRESHOLD\_MANUAL} = 65.0 \tag{4.2}$$

- $\geq 85.0 \rightarrow$ Firm warranty decision output
- $65.0 \leq \text{score} < 85.0 \rightarrow$ Conditional decision output
- $< 65.0 \rightarrow$ Needs Manual Review

Used in: Section 3.10.3 (Score Combination), Section 5.5 (Threshold Calibration)

5 Large Language Models (LLM Layer)

5.1 Transformer Architecture Overview

Large Language Models (LLMs) are neural networks trained on massive text corpora to predict the probability distribution of the next token in a sequence. The dominant architecture underlying modern LLMs is the Transformer, introduced by Vaswani et al. (2017). The Transformer forgoes recurrence in favour of a self-attention mechanism that allows every token in a sequence to attend to every other token simultaneously.

The core computation in a Transformer encoder layer is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{5.1}$$

where:

- Q = Query matrix
- K = Key matrix
- V = Value matrix
- d_k = key/query dimension

Multi-head attention is defined as:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W_O \tag{5.2}$$

$$\text{head}_i = \text{Attention}(QW_{Qi}, KW_{Ki}, VW_{Vi}) \tag{5.3}$$

TRACE does not implement a Transformer from scratch. Rather, it accesses pre-trained LLMs through a dual-provider architecture: OpenAI (gpt-4o-mini) as the primary provider and OpenRouter (arcee-ai/trinity-large-preview:free) as the fallback. Understanding the underlying architecture is relevant for interpreting the model's capabilities: the self-attention mechanism allows the LLM to resolve semantic ambiguities in technician notes by attending to contextual cues across the entire note, a capability that simple keyword matching cannot replicate.

Historical note: The original LLM integration used only OpenRouter. Dual-provider support (OpenAI + OpenRouter) was added in commit `d5a701d` ("Add dual-provider LLM support"). OpenAI is used when `OPENAI_API_KEY` is set; otherwise, OpenRouter is used as the fallback.

5.2 Prompt Engineering

Prompt engineering is the practice of carefully designing the natural-language input ("prompt") to an LLM to elicit a desired format, style, or content in its response. Because LLMs are trained to complete text, the phrasing, structure, and constraints embedded in the prompt strongly influence the output.

TRACE uses structured prompts with four key design patterns:

- **Explicit output format specification:** Each prompt instructs the model to respond only with JSON in an exact format.
- **Role framing:** Prompts begin with role definitions such as "You are an automotive warranty analyst."
- **Enumerated constraints:** Allowed values for categorical outputs are listed explicitly in the prompt to eliminate hallucinated outputs.
- **Disambiguation rules:** Stage 1 includes ordered disambiguation rules (first match wins) to resolve ambiguous categories (e.g., "overheating" → engine_symptom, not electrical_issue).

TRACE employs three distinct prompt templates: `UNDERSTAND_CLAIM_PROMPT`, `TRANSLATE_ML_FEATURES_PROMPT`, and `FORMAT_OUTPUT_PROMPT`.

LLM Configuration: All LLM calls use `temperature=0` and `seed=42` for deterministic outputs. The `response_format={"type": "json_object"}` parameter forces valid JSON output. Earlier versions used `temperature=0.3`, which produced inconsistent outputs; this was changed in commit `d851af8`.

5.3 Zero-Shot Inference

Zero-shot inference refers to using a pre-trained LLM to perform a task it was not explicitly fine-tuned for, relying solely on the instruction in the prompt. No labelled examples of the specific task are provided within the prompt.

TRACE uses zero-shot inference for all three LLM stages because:

1. the prompt constraints are sufficiently tight to guide the model reliably, and

2. including examples would increase per-request token cost and latency.

The deterministic temperature setting (`temperature=0`, `seed=42`) used in all TRACE LLM calls eliminates creative variation entirely, biasing the model towards the most likely and therefore most consistent JSON output.

5.4 API-Based LLM Integration

TRACE accesses LLM capabilities through a dual-provider architecture:

- **Primary:** OpenAI (gpt-4o-mini) when `OPENAI_API_KEY` is set.
- **Fallback:** OpenRouter (arcee-ai/trinity-large-preview:free) when only `OPENROUTER_API_KEY` is set.

The integration uses a standard HTTP POST request to the `/v1/chat/completions` endpoint (or the OpenAI client library), sending the prompt as a user-role message and specifying a JSON response format.

The LLM layer is entirely optional: if neither `OPENAI_API_KEY` nor `OPENROUTER_API_KEY` is set, or if the technician note is trivially short (≤ 5 characters), the pipeline bypasses all three LLM stages and falls back to deterministic feature extraction. This graceful degradation ensures the system remains operational in air-gapped or cost-sensitive deployments.

Retry logic with exponential back-off (2^{attempt} seconds, max 2 retries) is implemented in `understand_claim_with_retry()` to handle transient network failures and rate limiting.

LLM Categories (7 classes): The LLM categorizes technician notes into one of: `moisture_damage`, `physical_damage`, `ntf`, `electrical_issue`, `engine_symptom`, `communication_fault`, or `other`.

LLM Confidence Weight: When LLM signal is present, it contributes 15% weight (`LLM_WEIGHT = 0.15`) to the final combined confidence score. This was added in commit `d1904f2` ("integrate LLM confidence into combine_scores with 15% weight").

Used in: Section 3.10 (Six-Stage Pipeline), Section 5.6 (LLM Integration Testing)

6 Text Feature Extraction (Current Implementation)

6.1 TF-IDF Vectorisation of DTC Text

Term Frequency Inverse Document Frequency (TF-IDF) is a statistical measure that quantifies how characteristic a term is to a specific document relative to a corpus. It is the primary mechanism by which TRACE converts the raw DTC code string (e.g., "P0562 P0301") into a numerical feature vector for the ML classifier.

$$TF(t,d) = \frac{\text{count of } t \text{ in } d}{\text{total terms in } d} \tag{6.1}$$

$$IDF(t,D) = \log\left(\frac{1+|D|}{1+df(t)}\right) + 1 \tag{6.2}$$

$$TFIDF(t,d,D) = TF(t,d) \cdot IDF(t,D) \tag{6.3}$$

where:

- $|D|$ = total documents in corpus
- $df(t)$ = number of documents containing term t

In TRACE, each "document" is the space-joined DTC string for one claim, and the corpus is the full training set. The TF-IDF vectoriser is configured with `max_features=40`, retaining only the 40 most informative DTC token types across the corpus.

Critical: The TF-IDF vectoriser is fitted exclusively on the training split. At inference time, the stored vocabulary and IDF weights are applied via `.transform()`. Fitting on the full dataset before splitting would constitute data leakage.

Used in: Section 3.8.2 (Data Leakage), Section 5.1 (Feature Engineering)

6.2 Keyword-Based Complaint Matching (`match_complaint`)

In the current implementation, the technician's free-text notes are mapped to a fixed enumeration of 14 known complaint labels through a two-stage lookup. This mapping is performed by the `match_complaint()` function in `ml_predictor.py`.

Stage 1 Keyword Map: The function iterates over a dictionary of keywords and returns the first complaint whose keyword appears as a substring in the lowercased notes.

Stage 2 Fuzzy Fallback: If no keyword matches, `difflib.get_close_matches()` is called with a similarity cutoff of 0.25 to find the most similar complaint label in the known list. If no match exceeds the cutoff, the default label "OBD Light ON" is returned.

The resulting complaint label is one-hot encoded to produce a sparse binary feature vector. This feature column is the primary carrier of symptom information for the ML classifier.

Limitation: The fixed-vocabulary approach means that nuanced technician descriptions collapse to generic labels, losing contextual signal. The planned NLP enhancement described next addresses this directly.

6.3 High-Value DTC One-Hot Flags

Beyond TF-IDF, TRACE constructs explicit binary indicator features for a curated list of high-value DTC codes. For each code in the `HIGH_VALUE_DTCS` list, a binary feature `dtc_<code>` is 1 if that specific code is present in the claim's fault code string, and 0 otherwise.

This approach allows the ML classifier to exploit the strong warranty signal associated with specific DTC codes directly, without relying on the TF-IDF vocabulary to surface them with adequate weight.

Used in: Section 4.3 (Feature Engineering Details), Section 5.1 (Training)

7 Natural Language Processing Planned Enhancement

7.1 Motivation: Limitations of the Fixed Complaint Vocabulary

The current design constrains complaint input to 14 categorical labels. While this simplifies the ML feature space, it introduces two failure modes:

- **Information loss:** Technician language is rich in diagnostic nuance.
- **Mapping brittleness:** The keyword-to-complaint mapping is sensitive to vocabulary differences.

Replacing this categorical feature with a continuous semantic representation derived from the full text allows the model to leverage the complete information content of the technician's notes.

7.2 Text Preprocessing and Tokenisation

Before any vectorisation, raw technician notes must be preprocessed to reduce noise and normalise vocabulary. Standard preprocessing steps include:

- Lowercasing
- Punctuation removal
- Stop word removal
- Tokenisation
- Stemming / Lemmatisation

7.3 Vectorisation Approaches

7.3.1 TF-IDF on Complaint Text

The simplest NLP enhancement replaces the one-hot encoded complaint label with a TF-IDF vector computed directly from the raw technician notes. Using a vocabulary of 100–300 terms, this yields a sparse feature vector that captures word importance across the training corpus.

7.3.2 Dense Sentence Embeddings

A more powerful approach uses pre-trained sentence embedding models to encode each complaint note as a fixed-length dense vector:

$$e = \text{SentenceEncoder}(\text{text}) \tag{7.1}$$

Cosine similarity between two embeddings is:

$$\cos(e_1, e_2) = \frac{e_1 \cdot e_2}{\|e_1\|\|e_2\|} \tag{7.2}$$

Dense embeddings can represent semantically similar complaints as nearby vectors, even when they share no common keywords.

7.3.3 LLM-Based Semantic Classification (Current Partial Implementation)

TRACE already implements a form of NLP-driven complaint understanding through the LLM layer. When the OpenRouter API is available, `understand_claim()` semantically categorises the notes and returns a normalised complaint string alongside related fields.

The planned enhancement formally integrates this semantic output as a persistent ML feature by generating dataset-level semantic complaint features during training.

7.4 Integration Architecture for Free-Text NLP

Regardless of the chosen vectorisation strategy, the NLP layer integrates into the TRACE pipeline as a feature transformation step:

$$\text{Technician Notes} \rightarrow [\text{NLP Vectoriser}] \rightarrow X_{\text{complaint\_nlp}} \tag{7.3}$$

$$X_{\text{base}} = [X_{\text{complaint\_nlp}} \mid X_{\text{dtc\_tfidf}} \mid X_{\text{dtc\_flags}} \mid \cdots] \tag{7.4}$$

$$\rightarrow \texttt{clf\_fa.predict()} + \texttt{clf\_wd.predict()} \tag{7.5}$$

The NLP vectoriser object must be persisted alongside the trained models in the model bundle and called at inference time to transform incoming free-text notes before feature assembly.

Used in: Section 6.2 (Future Work — NLP Enhancement)

8 Data Preprocessing and Feature Engineering

8.1 One-Hot Encoding (OHE)

One-hot encoding converts a categorical variable with k distinct values into a binary vector of length k, where exactly one position is 1 and all others are 0.

In TRACE, `OneHotEncoder` is applied to:

- Customer Complaint
- Supplier
- Mileage Bracket
- Voltage Bracket
- DTC Count Bracket

All OHE instances are fitted on the training split only and serialised into the model bundle for consistent application at inference time. `handle_unknown="ignore"` is set to gracefully handle unseen categories.

8.2 Standard Scaling

Standard scaling transforms a continuous feature to have zero mean and unit variance:

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}} \tag{8.1}$$

where μ_{train} and σ_{train} are computed on the training split only.

In TRACE, `StandardScaler` is applied to `Mileage_km`, `Year`, `Claim Age`, and `Voltage` for consistency with the sparse matrix pipeline and forward compatibility.

8.3 Mileage Binning and Bracket Encoding

Raw odometer readings are discretised into brackets:

- $[0, 20000) =$ `low`
- $[20000, 60000) =$ `mid`
- $[60000, 100000) =$ `high`
- $[100000, \infty) =$ `very_high`

This mileage_bracket feature is derived after the train/test split and then one-hot encoded.

8.4 Voltage Binning and Bracket Encoding

Voltage readings are discretised into brackets based on critical automotive thresholds:

- $[0, 11.0)$ = `under_voltage` (battery depleted)
- $[11.0, 15.4)$ = `normal` (normal operating range)
- $[15.4, 17.0)$ = `high` (EOS threshold for ASICs)
- $[17.0, \infty)$ = `extreme` (severe over-voltage)

This voltage_bracket feature captures non-linear voltage effects that a single scaled numeric would miss.

8.5 DTC Count Bracket Encoding

The number of DTC codes in a claim is discretised:

- 0 = `none`
- 1 = `single`
- 2–3 = `few`
- 4+ = `many`

Customer Failure claims average 2.3 DTC codes vs. 1.5 for Production Failure, making this a predictive signal.

8.6 Interaction Features

Cross-terms between voltage and DTC prefix capture failure modes that neither feature alone would detect:

- `volt_high_and_P`: (Voltage $>$ 15.4) $\wedge$ (has_P $=$ 1) — high voltage + P-code = strong PF signal
- `volt_low_and_U`: (Voltage $<$ 11.0) $\wedge$ (has_U $=$ 1) — low voltage + U-code = network fault
- `volt_normal_and_C`: (11 $\leq$ Voltage $\leq$ 14.5) $\wedge$ (has_C $=$ 1) — normal voltage + C-code = chassis issue
- `has_multiple_prefixes`: (has_P + has_U + has_C + has_B) $>$ 1 — multiple DTC categories $\rightarrow$ more severe

8.7 Claim Age Derivation

Claim Age is computed as:

$$\text{claim\_age} = \text{claim\_date.year} - \text{vehicle\_manufacture\_year} \tag{8.2}$$

This feature directly encodes warranty eligibility and has high practical value.

8.8 Sparse Matrix Concatenation

The TRACE feature matrix is assembled from a heterogeneous mix of dense arrays and sparse matrices. All components are combined using `scipy.sparse.hstack()`:

$$X = \texttt{hstack}([X_{\text{complaint}}, X_{\text{dtc\_tfidf}}, X_{\text{dtc\_flags}}, X_{\text{supplier}}, X_{\text{mileage}}, X_{\text{year}}, X_{\text{mileage\_bracket}}, X_{\text{claim\_age}}, X_{\text{voltage\_bracket}}, X_{\text{dtc\_count\_bracke} \tag{8.3}$$

The resulting sparse matrix X is passed directly to the classifiers.

Used in: Section 5.1 (Feature Matrix Assembly), Section 3.8.1 (Train/Test Split)

9 Training Pipeline Architecture

9.1 Train/Test Split

The dataset is divided into a training set (80%) and a held-out test set (20%) using `train_test_split()` with a fixed `random_state=42`.

$$\text{Train} = 80{,}000 \text{ samples} \tag{9.1}$$

$$\text{Test} = 20{,}000 \text{ samples} \tag{9.2}$$

All feature transformers are fitted using only the training split and then applied to both training and test sets.

9.2 Data Leakage and Prevention

Data leakage occurs when information from the test set is inadvertently used during model training, causing inflated evaluation metrics.

Two forms of leakage are particularly relevant to TRACE:

- **Transformer leakage**
- **Cascade leakage**

Rule: Fit on train, transform on test.

9.3 Out-of-Fold Cross-Validation for Cascade Training

K-fold cross-validation partitions the training set into K equal-sized folds. For each of K iterations, $K-1$ folds are used for training and the remaining fold is used for evaluation.

$$\text{For } k = 1 \text{ to } K: \tag{9.3}$$

$$\quad \text{Train } \texttt{clf\_fa} \text{ on folds } \{1, \ldots, K\} \setminus \{k\} \tag{9.4}$$

$$\quad \text{Predict probabilities on fold } k \tag{9.5}$$

TRACE uses `cross_val_predict()` with `cv=5` and `method="predict_proba"` to generate OOF FA probabilities.

9.4 Model Serialisation with Pickle

Once training is complete, all model objects and fitted transformers are bundled into a Python dictionary and serialised to disk using Python's `pickle` module.

```
bundle = {
  'clf_fa': clf_fa,                    # XGBClassifier (6 classes)
  'clf_wd': clf_wd,                    # XGBClassifier (3 classes)
  'le_fa': le_fa,                      # LabelEncoder (FA)
  'le_wd': le_wd,                      # LabelEncoder (WD)
  'ohe': ohe,                          # OneHotEncoder (Customer Complaint)
  'tfidf_d': tfidf_d,                  # TfidfVectorizer (DTC text, max_features=40)
  'ohe_supplier': ohe_supplier,        # OneHotEncoder (Supplier)
  'mileage_scaler': mileage_scaler,    # StandardScaler (Mileage_km)
  'year_scaler': year_scaler,          # StandardScaler (Year)
  'ohe_mileage': ohe_mileage,          # OneHotEncoder (mileage_bracket)
  'claim_age_scaler': claim_age_scaler,# StandardScaler (Claim Age)
  'voltage_scaler': voltage_scaler,    # StandardScaler (Voltage)
  'ohe_voltage_bracket': ohe_voltage_bracket,  # OneHotEncoder (voltage_bracket)
  'ohe_dtc_count_bracket': ohe_dtc_count_bracket  # OneHotEncoder (dtc_count_bracket)
}
```

Historical note: The original bundle contained RandomForest classifiers and fewer transformers (no voltage_scaler, ohe_voltage_bracket, ohe_dtc_count_bracket). The bundle was expanded when voltage features and DTC count brackets were added to the ML model.

Used in: Section 5.1 (Training and Saving), Section 6.1 (Deployment)

10 Model Evaluation Metrics

10.1 Confusion Matrix

The confusion matrix is a square matrix of dimensions $C \times C$, where C is the number of classes. Entry (i, j) represents the number of instances where the true label was class i and the predicted label was class j.

For binary classification, the four cells are:

Cell	Meaning
True Positive (TP)	Predicted positive, actual positive — correct approval
True Negative (TN)	Predicted negative, actual negative — correct rejection
False Positive (FP)	Predicted positive, actual negative — wrongly approved
False Negative (FN)	Predicted negative, actual positive — wrongly rejected

10.2 Accuracy

$$\text{Accuracy} = \frac{\text{number of correct predictions}}{\text{total predictions}} \tag{10.1}$$

Accuracy is useful when classes are balanced.

10.3 Precision

$$\text{Precision}(c) = \frac{TP_c}{TP_c + FP_c} \tag{10.2}$$

High precision means that when the model predicts class c, it is usually right.

10.4 Recall (Sensitivity)

$$\text{Recall}(c) = \frac{TP_c}{TP_c + FN_c} \tag{10.3}$$

High recall means that the model correctly finds most true instances of class c.

10.5 F1 Score

$$F1(c) = \frac{2 \cdot \text{Precision}(c) \cdot \text{Recall}(c)}{\text{Precision}(c) + \text{Recall}(c)} \tag{10.4}$$

Equivalently,

$$F1 = \frac{2TP}{2TP + FP + FN} \tag{10.5}$$

10.6 Macro vs Weighted Averaging

$$\text{Macro F1} = \frac{1}{C} \sum_{c=1}^{C} F1(c) \tag{10.6}$$

$$\text{Weighted F1} = \sum_{c=1}^{C} \left(\frac{\text{support}(c)}{N} \right) F1(c) \tag{10.7}$$

Macro averaging treats all classes equally, while weighted averaging accounts for class frequency.

10.7 Classification Report

The `classification_report()` function generates a tabular summary presenting precision, recall, F1 score, and support for every class, followed by macro-averaged and weighted-averaged summary rows.

Used in: Section 5.4 (Evaluation Section), Section 5.5 (Results Discussion)

11 Hybrid Pipeline Architecture

11.1 What is a Hybrid ML Pipeline?

A hybrid ML pipeline is a system that combines two or more fundamentally different inference mechanisms — such as deterministic rule engines, statistical classifiers, and neural language models — into a unified decision process.

The TRACE pipeline is hybrid in three senses:

1. it combines rule-based and ML-based inference,
2. it operates conditionally, and
3. its final output is produced by blending confidence scores from multiple components.

11.2 Six-Stage Prediction Flow

The `predict()` function in `ml_predictor.py` orchestrates a six-stage pipeline:

Stage	Component	Description
1	LLM Understanding (optional)	`understand_claim_with_retry()` categorises the claim into 7 categories and extracts an initial failure analysis hypothesis. Dual-provider: OpenAI (gpt-4o-mini) or OpenRouter fallback.
2	Rule Engine	`run_rules()` evaluates 9 priority-ordered rules (over_voltage, low_voltage, moisture, physical_damage, ntf, u_code, p_code_engine, c_code, b_code).
3	Feature Translation	Builds the structured feature dictionary. LLM path: `translate_to_ml_features()`. Fallback: `extract_dtc_features()` + `match_complaint()`.
4	XGBoost Cascade Scoring	Runs FA classifier (6 classes), augments features with FA probabilities, then runs WD classifier (3 classes).
5	Score Combination	Blends rule, ML, and LLM confidence with agreement-weighted coefficients. LLM contributes 15% weight.
6	Output Formatting	Assembles the final API response dictionary. LLM path: `format_output()`. Fallback: `assemble_output_from_fields()`.

11.3 Score Combination and Weighted Blending

The `combine_scores()` function implements weighted blending with three components: rule confidence, ML confidence, and LLM signal:

$$\text{If rule fires and agrees with ML:} \quad \text{combined} = 0.70 \cdot \text{rule\_conf} + 0.30 \cdot \text{ml\_conf} + 5.0 \tag{11.1}$$

$$\text{If rule fires and disagrees:} \quad \text{combined} = 0.55 \cdot \text{rule\_conf} + 0.35 \cdot \text{ml\_conf} \tag{11.2}$$

With LLM signal present: When the LLM is available and its confidence exceeds the low-confidence threshold (0.3), the LLM contributes 15% weight:

$$\text{combined} = 0.595 \cdot \text{rule\_conf} + 0.255 \cdot \text{ml\_conf} + 0.15 \cdot (\text{llm\_conf} \times 100) + 5.0 \tag{11.3}$$

If no rule fires:

$$\text{combined} = \text{ml\_conf} \cdot (1 - \text{LLM\_WEIGHT}) + \text{LLM\_WEIGHT} \cdot (\text{llm\_conf} \times 100) \tag{11.4}$$

If the LLM flagged input as "other" with low confidence (< 0.3), the score is penalised:

$$\text{combined} = 0.85 \times \text{combined} \tag{11.5}$$

Final score:

$$\text{combined} = \text{clamp}(\text{round}(\text{combined}, 1), 0.0, 98.0) \tag{11.6}$$

Weight constants:

- `RULE_WEIGHT_AGREE` = 0.70
- `ML_WEIGHT_AGREE` = 0.30
- `LLM_WEIGHT` = 0.15

- `RULE_WEIGHT_DISAGREE` = 0.55
- `ML_WEIGHT_DISAGREE` = 0.35
- `AGREEMENT_BONUS` = 5.0
- `WEAK_INPUT_PENALTY` = 0.85

11.4 Confidence Thresholds and Status Assignment

- combined $\geq 85.0 \rightarrow$ Firm decision
- $65.0 \leq$ combined $< 85.0 \rightarrow$ Conditional decision
- combined $< 65.0 \rightarrow$ Needs Manual Review

The manual review path is an intentional safety mechanism.

11.5 Decision Engine Tagging

Every TRACE output includes a `decision_engine` field indicating which layers contributed to the final decision:

- `LLM+Rule+ML` — all three layers contributed
- `Rule+ML` — rule fired, no LLM signal
- `LLM+ML` — no rule fired, LLM present
- `ML` — neither rule nor LLM contributed

Tracking decision engine distribution in production telemetry supports auditing and future rule-engine expansion.

Used in: Section 5.6 (Production Monitoring), Section 6.1 (Future Enhancements)

12 Domain Concepts: Automotive Warranty Classification

12.1 Diagnostic Trouble Codes (DTCs)

A Diagnostic Trouble Code (DTC) is a standardised alphanumeric code stored by an automotive Electronic Control Unit (ECU) when the vehicle's onboard diagnostics system detects a fault condition.

Each DTC follows the format:

[Prefix][System][Fault Number]

The TRACE dataset contains DTCs from over 1,973 distinct codes covering Indian automotive OEM vehicles.

12.2 DTC Prefix Taxonomy (P, U, C, B)

12.3 Failure Analysis Categories

Failure Analysis (FA) is the root cause classification of the warranty claim. TRACE predicts categories such as:

- NTF (No Trouble Found)

Prefix	System	Examples	Typical Warranty Signal
P	Powertrain	P0300, P0562	Strong production failure indicator when combined with engine symptoms
U	Network	U0100, U0073	High production failure probability; communication faults rarely caused by customer
C	Chassis	C0045, C0265	Moderate production failure probability
B	Body	B1234, B2960	Variable; depends on physical damage evidence

- Sensor short due to moisture
- Connector damage
- ASIC CJ327 failure due to EOS
- Controller failure due to supplier production failure
- Injector failure
- Wiring harness failure

12.4 Warranty Decision Types

Decision	Meaning	Typical Action
Production Failure	Fault attributable to manufacturing or supplier defect	Claim Approved
Customer Failure	Fault caused by misuse, accident, environmental exposure, or modification	Claim Rejected
According to Specification	Vehicle/component operating correctly; no defect exists	Claim Closed

TRACE's hybrid confidence scoring and manual review fallback are designed specifically to minimise costly approval and rejection errors.

Used in: Section 2.3 (Problem Statement), Section 5.5 (Results Discussion)

13 Chapter Summary

The following table provides a cross-reference between each foundational concept and the TRACE component where it is applied.
End of Chapter 3.

Concept	**TRACE Artefact / Section**
Supervised Multi-class Classification	Both XGBoost classifiers (`clf_fa`, `clf_wd`) in `ml_predictor.py`
XGBoost (1000 estimators)	`clf_fa` and `clf_wd` in `train_and_save()`; replaced RandomForest (commit `a518371`)
Cascade Classification	`fa_probs` appended to `X_wd`; OOF via `cross_val_predict()`
Rule Engine (9 rules)	`RULES` list + `run_rules()` in `ml_predictor.py`; over_voltage, low_voltage added
Confidence Thresholding	`combine_scores()` — 85% firm, 65% manual, below 65% review
LLM Confidence Weight	`LLM_WEIGHT = 0.15` in `combine_scores()` (commit `d1904f2`)
Dual LLM Providers	OpenAI (gpt-4o-mini) + OpenRouter fallback in `llm_client.py`
LLM Consistency	`temperature=0`, `seed=42`, disambiguation rules (commit `d851af8`)
Prompt Engineering	Prompt templates for understand, translate, and output formatting
Zero-shot Inference	Stage 1 and Stage 3 LLM calls
TF-IDF Vectorisation	`tfidf_d` on DTC text (max_features=40)
Keyword Complaint Matching	`match_complaint()` in `ml_predictor.py`
High-Value DTC One-Hots	`HIGH_VALUE_DTCS` list (∼90 codes) as binary features
One-Hot Encoding	Complaint, supplier, mileage bracket, voltage bracket, DTC count bracket OHE
Standard Scaling	Mileage, year, claim age, voltage scalers
Mileage Binning	`pd.cut()` + mileage bracket OHE
Voltage Binning	voltage_bracket OHE (under_voltage, normal, high, extreme)
DTC Count Binning	dtc_count_bracket OHE (none, single, few, many)
Interaction Features	volt_high_and_P, volt_low_and_U, volt_normal_and_C, has_multiple_prefixes
Claim Age Feature	Derived from claim date and vehicle year
Sparse Matrix Concatenation	`scipy.sparse.hstack()`
Train/Test Split	`train_test_split(test_size=0.2, random_state=42)`
Data Leakage Prevention	Fit on train only; OOF for cascade
Out-of-Fold Cross-Validation	`cross_val_predict(cv=5, method='predict_proba')`
Model Serialisation	`pickle.dump(bundle)` → `trace_models.pkl`
Accuracy, Precision, Recall, F1	`evaluate_model.py`; `classification_report()`
Hybrid Pipeline	`predict()` orchestrator in `ml_predictor.py`
Score Combination	`combine_scores()` with rule/ML/LLM weights
Graceful Degradation	LLM fallback paths in `predict()` (Rule+ML or ML mode)
DTC Prefix Taxonomy	Prefix-based features + rule engine
Failure Analysis Labels	`le_fa` + 6-class classifier output
Warranty Decision Labels	`le_wd` + 3-class classifier output
EasyOCR Integration	`/scan-image-easyocr` endpoint in `main.py` for DTC/voltage extraction from images
Centralized Logging	`logging_config.py` with DecisionLogger across all pipeline stages